

Hands-On Computer Vision and Prosthetic Control: An Intermediate Workshop

Severus Prosthetics Workshop
Motion Recognition Workshop Series

Abstract—This workshop teaches two linked skills in roughly one hour: real-time computer-vision hand tracking, and translating that tracking into control of a prosthetic hand. Participants extend a working program by completing five short edits, each of which switches on one stage of the pipeline – detection, visualisation, per-finger state estimation, and actuator command mapping. The program runs at every stage, degrading gracefully so unfinished features are simply labelled on screen. This intermediate edition asks participants to write each value themselves, supported by technical tips that make external references unnecessary.

Index Terms—computer vision, MediaPipe, hand tracking, prosthetics, human-machine interface, gesture recognition, education

I. INTRODUCTION

Modern prosthetic hands increasingly take their commands from sensors that estimate user intent. A camera plus a hand-tracking model is one of the most accessible such sensors: it needs no electrodes and runs on a laptop. This workshop builds that path end to end, from pixels to finger motion, using the Severus hand and its simulator.

The exercise file opens a live window immediately. Five marked lines (named ANSWER_1 through ANSWER_5) each enable one stage. As you complete them, the program progresses from doing very little, to drawing the hand, to recognising finger states, to moving a simulated (and optionally physical) prosthesis.

II. BACKGROUND

MediaPipe estimates 21 landmarks per hand. Each landmark has normalised image coordinates, where x increases to the right and y increases DOWNWARD from the top of the frame. A finger is treated as extended when its fingertip is higher than the knuckle below it – a smaller y. The thumb is handled on the x axis instead, because it moves sideways.

These per-finger booleans become commands. In simulation each finger has an openness between 0.0 (closed) and 1.0 (open); the identical command stream drives the physical Severus hand once it is connected. The workshop therefore mirrors a real human-machine interface: sense, interpret, actuate.

III. WORKSHOP EXERCISES

A. Exercise 1: Hand-landmark detection

Line to edit: ANSWER_1.

Replace the placeholder with your own value:

```
ANSWER_1 = None          # choose the correct vision
                        task class
```

What this line does. Selects the MediaPipe vision task that, for every frame, returns the 21 hand keypoints (knuckles and fingertips) as normalised coordinates. The program then builds its detector from this class.

Tip. The vision task class returning 21 HAND landmarks (Hand + Landmarker).

Why it matters for control. This is the SENSING front end of the prosthesis: it converts a raw camera image into structured numbers. Without a reliable hand estimate there is no signal to map onto the fingers, so every later step depends on this one.

B. Exercise 2: Drawing the landmarks

Line to edit: ANSWER_2.

Replace the placeholder with your own value:

```
ANSWER_2 = None          # choose the function that
                        draws dots + bones
```

What this line does. Picks the drawing function that overlays the 21 keypoints and the skeleton connections onto the camera frame so the tracked hand is visible.

Tip. A function reference (no parentheses) drawing dots+bones, not text.

Why it matters for control. Visual feedback is how an engineer validates a sensor before trusting it for control. Seeing the skeleton lock onto the hand confirms the estimate is stable enough to drive an actuator.

C. Exercise 3: Finger extend / retract rule

Line to edit: ANSWER_3.

Replace the placeholder with your own value:

```
ANSWER_3 = None          # comparison operator as a
                        string: "<" or ">"
```

What this line does. Defines, for the four fingers, the rule that turns two landmark heights into a binary state. The engine evaluates tip-y (operator) pip-y; you supply the operator. Image coordinates put y = 0 at the TOP and increase downward, so a raised (extended) fingertip has the SMALLER y.

Tip. Image y grows downward; an extended fingertip has the smaller y.

Why it matters for control. This is the heart of joint-space mapping: a continuous measurement is thresholded into

an open/close command per finger. The same idea scales up to proportional control, but binary is the clearest start.

D. Exercise 4: Thumb extend / retract rule

Line to edit: ANSWER_4.

Replace the placeholder with your own value:

```
ANSWER_4 = None          # comparison operator as a
string: "<" or ">"
```

What this line does. Defines the thumb rule. Because the thumb abducts SIDEWAYS rather than up and down, it is classified on the x axis: the engine evaluates tip-x (operator) ip-x for a right hand and mirrors it automatically for a left hand.

Tip. Right-hand extended thumb: tip x is greater than ip x.

Why it matters for control. Real hands are not uniform; the thumb has a different kinematic axis. Treating it separately mirrors how prosthetic controllers special-case the thumb, which dominates grasp quality.

E. Exercise 5: Driving the simulated hand

Line to edit: ANSWER_5.

Replace the placeholder with your own value:

```
ANSWER_5 = None          # openness float for an
EXTENDED finger (0.0..1.0)
```

What this line does. Connects the detected states to the actuator model. Each simulated finger takes an openness value in the range 0.0 to 1.0; the engine uses ANSWER_5 for an extended finger and one-minus-ANSWER_5 for a retracted one, then eases smoothly toward that target.

Tip. The openness value for a fully EXTENDED/open finger, in [0.0, 1.0].

Why it matters for control. This is the ACTUATOR command mapping. The very same per-finger command stream drives the on-screen hand and, once an Arduino is connected, the physical Severus prosthesis – simulation and hardware share one interface.

IV. CONCLUSION

Completing the five edits yields a full sense-interpret-actuate loop: a camera tracks the hand, per-finger states are inferred, and a prosthetic hand mirrors the motion. The same structure underlies production myoelectric and vision-based controllers; only the sophistication of each stage grows. If a finished file is connected to a prepared Severus hand, it will move the real fingers exactly as it moves the simulated ones.